



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

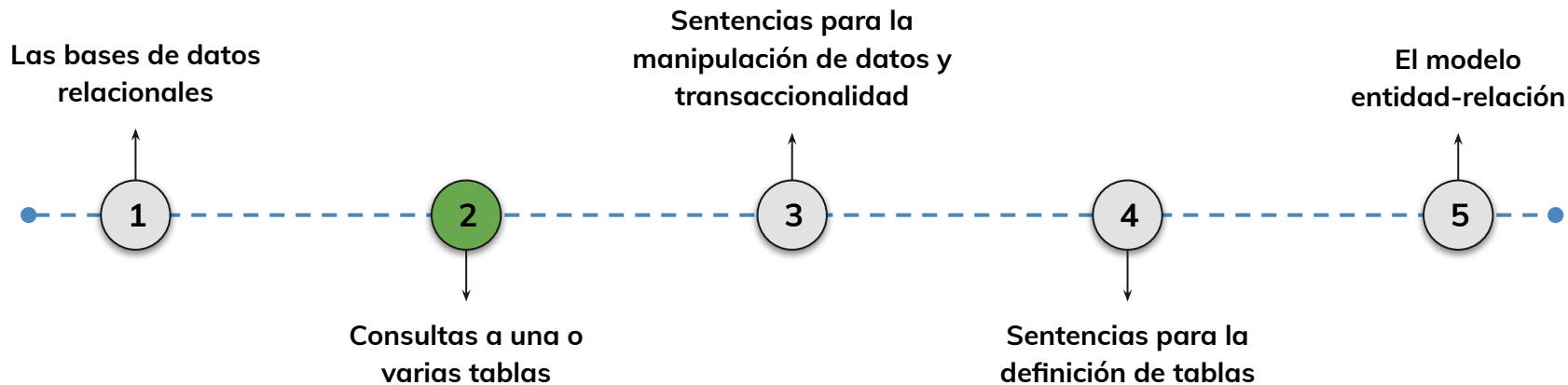


› Consultas a una o varias tablas - Parte 2

AE2: Utilizar lenguaje estructurado de consultas sql para la obtención de información que satisface los requerimientos planteados a partir de un modelo de datos dado.

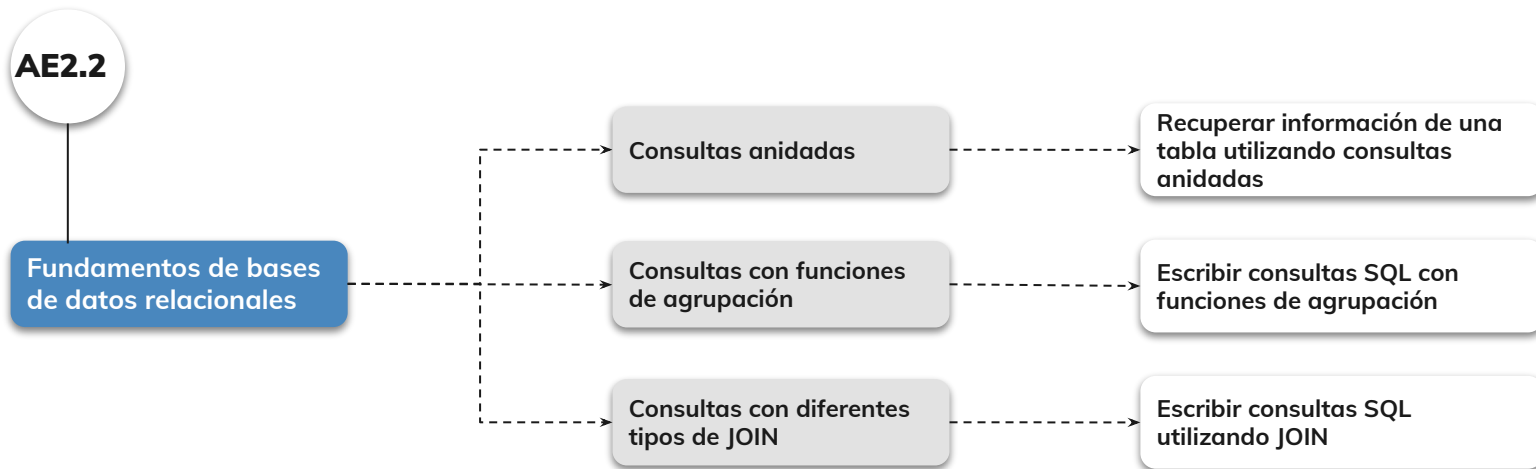
Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?





Objetivos de aprendizaje

¿Qué aprenderás?



- Comprender qué son las consultas anidadas
- Realizar consultas anidadas utilizando SQL
- Realizar consultas con funciones de agrupación utilizando SQL
- Realizar consultas con distintos tipos de JOIN

Repaso clase anterior

¿Quedó alguna duda?

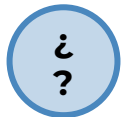
En la clase anterior trabajamos :

- Definir el concepto SQL
- Recuperar información de una tabla utilizando SQL
- Diferenciar los distintos tipos de cláusulas SQL que se utilizan para recuperar datos de una tabla
- Comprender qué es un modelo de datos
- Aprender la definición de una llave primaria
- Aprender la definición de una llave llave foránea/externa
- Realizar consultas SQL utilizando claves primarias y foráneas

#Momentode Preguntas...



¿Qué son las consultas anidadas?



¿Qué son las funciones de agregación y para qué sirven?



¿Qué son las operaciones JOIN?

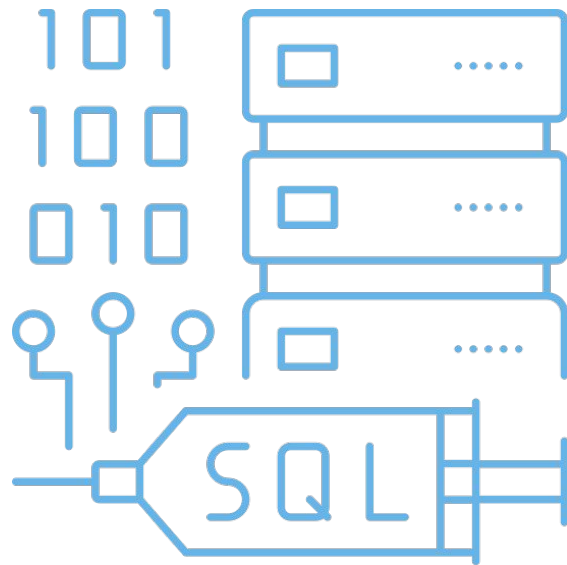
Consultas anidadas

Consultas anidadas

¿Qué son las consultas anidadas?:

Las consultas anidadas en SQL, también conocidas como **subconsultas o subqueries**, son consultas que se encuentran dentro de otra consulta principal. Estas subconsultas se utilizan para realizar operaciones más complejas al **combinar datos de múltiples tablas o para filtrar datos** de manera más precisa utilizando los resultados de una consulta interna en la consulta principal.

En una consulta anidada, la consulta interna (subconsulta) se ejecuta primero y luego su resultado se utiliza en la consulta externa (consulta principal).





Consultas anidadas

Analicemos la cláusula `SELECT`



En SQL, las cláusulas SELECT, FROM y WHERE son **componentes fundamentales de una consulta** que te permiten recuperar y filtrar datos de una base de datos.

Cláusula SELECT:

Se utiliza para especificar **qué columnas o expresiones deseas recuperar** de la base de datos. Define las columnas que estarán presentes en los resultados de la consulta.

Veamos un ejemplo en donde la consulta seleccionará las columnas "nombre" y "edad" de la tabla "usuarios":

```
SELECT nombre, edad FROM usuarios;
```



Consultas anidadas

Analicemos la cláusula **FROM**



Cláusula FROM:

Se utiliza para indicar **de qué tabla o tablas deseas recuperar los datos.** Especifica la fuente de donde se obtendrán los datos que se procesarán en la consulta. Puedes incluir una o varias tablas separadas por comas, y si hay más de una tabla, puedes utilizar la cláusula JOIN para combinarlas según ciertas condiciones.

Veamos un ejemplo en donde la consulta recupera los nombres de los clientes y los nombres de los productos de la tabla "pedidos"

```
SELECT nombre, producto  
FROM pedidos
```



Consultas anidadas

Analicemos la cláusula **WHERE**



Cláusula WHERE:

Se utiliza para aplicar **condiciones de filtro a los datos que se están recuperando**. Solo se seleccionarán los registros que cumplan con las condiciones especificadas en esta cláusula.

Puedes utilizar operadores de comparación y lógicos para definir estas condiciones.

Veamos un ejemplo en donde la consulta seleccionará los nombres y edades de los usuarios de la tabla "usuarios" donde la edad sea mayor que 25:

```
SELECT nombre, edad  
FROM usuarios  
WHERE edad > 25;
```

Consultas anidadas

¿Cómo lucen una consulta anidada?

Las subconsultas pueden aparecer en varias partes de una consulta principal, como en la cláusula SELECT, FROM, WHERE, HAVING o incluso en la cláusula JOIN.

El resultado de la subconsulta se utiliza como valor en la consulta principal.

Ejemplo:

```
SELECT nombre
FROM empleados
WHERE salario > (SELECT AVG(salario) FROM salarios);
```

Analicemos el ejemplo:

Hay dos tablas: "empleados" y "salarios".

Queremos encontrar los nombres de los empleados cuyos salarios sean superiores al promedio de todos los salario:

- 1) La **subconsulta** (SELECT AVG(salario) FROM salarios) calcula el promedio de todos los salarios en la tabla "salarios".
- 2) La **consulta principal** selecciona los nombres de los empleados cuyos salarios sean mayores que ese promedio.



Consultas anidadas



A continuación tenemos otro ejemplo:

Supongamos que tienes una base de datos de estudiantes y cursos, y deseas encontrar el nombre de los estudiantes que están inscritos en el curso llamado "Matemáticas Avanzadas".

```
SELECT nombre
FROM Estudiantes
WHERE id_estudiante IN (
    SELECT id_estudiante
    FROM Inscripciones
    WHERE id_curso = (SELECT id_curso FROM Cursos WHERE nombre_curso =
        'Matemáticas Avanzadas')
);
```



Consultas anidadas



¡Analicemos el ejemplo anterior!

En este ejemplo, la subconsulta interna obtiene el `id_curso` correspondiente al curso "Matemáticas Avanzadas".

Luego, la subconsulta principal utiliza este `id_curso` para recuperar los `id_estudiante` de los estudiantes inscritos en ese curso.

Por último:

Las consultas anidadas son especialmente útiles cuando necesitas realizar operaciones basadas en resultados previos o cuando deseas aplicar condiciones más precisas a tus búsquedas.

LIVE CODING

Recuperar información de una tabla utilizando consultas anidadas(parte 1):

A partir de una base de datos que almacena información sobre empleados y departamentos en una empresa. Tienes dos tablas: "Empleados" y "Departamentos" con las siguientes columnas:

Tabla "Empleados":

```
id empleado (Clave primaria)
nombre
apellido
salario
id_departamento
```

Tabla "Departamentos":

```
id departamento (Clave primaria)
nombre departamento
ubicacion
```

Tiempo: 25 min.

LIVE CODING

Realizar las siguientes tareas (parte 2):

1. Obtener el nombre y apellido de los empleados que trabajan en el departamento con nombre "Ventas".
2. Obtener los nombres de los empleados y sus salarios que trabajan en el mismo departamento que el empleado con "id_empleado" igual a 2.
3. Recupera el nombre del empleado cuya ubicación sea igual a Chile
4. Recupera los nombres de los empleados cuyo "id_departamento" sea igual a 1.

Tiempo: 25 min.



Ejercicio N° _1_

Consultas anidadas (Sub-queries)

Ejercicio guiado – Consultas anidadas (Sub-queries)

Contexto: 🙌

Hasta este punto de la clase vimos

- Subconsultas simples con IN y =.
- Subconsultas correlacionadas.
- Uso de funciones de agregación dentro de subconsultas.

Ahora combinaremos esas ideas en un reto único de 3 partes.

Consigna: 📝

Trabajaremos con las tablas que ya venimos usando (Libros, Generos, Autores). Para ello tendrán que resolver cada ítem sólo con subconsultas (¡no JOINS!) que incluyan un SELECT final para mostrar resultados.

Ejercicio guiado – Consultas anidadas (Sub-queries)

1. Género menos frecuente

- Identifiquen cuál es el id_genero con menor cantidad de libros en la tabla Libros.
- Muestren todos los títulos que pertenecen a ese género.

2. Autores “prolíficos recientes”

- Calculen el año de publicación promedio de toda la biblioteca (subconsulta agregada).
- Devuelvan nombre, apellido de los autores cuya fecha promedio de publicación sea posterior a ese promedio global. (Subconsulta correlacionada sobre Libros.)

3. Top-3 géneros por cantidad de libros

- Mediante una subconsulta con ORDER BY ... LIMIT 3, obtengan los tres géneros con más libros.
- Listen nombre_genero y el conteo de libros de cada uno.

Tiempo 🕒: 20 minutos



Momento:

Time-out!

 5 -10 min.



Consultas con funciones de agrupación

Consultas con funciones de agrupación

¿Qué son y para qué sirven?

Las consultas con funciones de agregación en SQL son **utilizadas para calcular valores** resumidos o agregados, como sumas, promedios, máximos, mínimos y conteos, sobre un conjunto de datos. Estas funciones permiten realizar cálculos en grupos de registros en lugar de tratar cada registro individualmente.

Si se utilizan junto con la cláusula **GROUP BY** para agrupar los registros según una o varias columnas y luego aplicar las funciones de agregación a cada grupo.

```
-- Contar el número de empleados por departamento
SELECT id_departamento, COUNT(*) AS cantidad_empleados
FROM Empleados
GROUP BY id_departamento;
```

Consultas anidadas



Las funciones de agregación más comunes en SQL son:

- **SUM:** Calcula la suma de los valores de una columna numérica en el grupo.
- **AVG:** Calcula el promedio de los valores de una columna numérica en el grupo.
- **COUNT:** Cuenta el número de registros en el grupo. Puede contar todos los registros o solo los registros no nulos en una columna específica.
- **MAX:** Devuelve el valor máximo de una columna en el grupo.
- **MIN:** Devuelve el valor mínimo de una columna en el grupo.

Veamos algunos ejemplos en la siguiente diapositiva



Consultas anidadas

Ejemplos de consultas con funciones de agregación



```
-- Calcular la suma de los salarios de todos los empleados
SELECT SUM(salario) AS suma_salarios
FROM Empleados;

-- Calcular el promedio de los salarios por departamento
SELECT id_departamento, AVG(salario) AS promedio_salarios
FROM Empleados
GROUP BY id_departamento;

-- Contar el número de empleados por departamento
SELECT id_departamento, COUNT(*) AS cantidad_empleados
FROM Empleados
GROUP BY id_departamento;

-- Encontrar el salario máximo y mínimo por departamento
SELECT id_departamento, MAX(salario) AS salario_maximo, MIN(salario) AS salario_minimo
FROM Empleados
GROUP BY id_departamento;
```



Consultas anidadas

Aquí se pueden ver varias funciones de agregación en una misma consulta. Junto a una primera aparición de **JOIN**

```
SELECT
  d.nombre_departamento,
  COUNT(e.id_empleado)      AS n_empleados,
  ROUND(AVG(e.salario), 2)   AS salario_prom,
  MIN(e.salario)             AS salario_min,
  MAX(e.salario)             AS salario_max,
  SUM(e.salario)             AS masa_salarial
FROM Departamentos d
LEFT JOIN Empleados e ON e.id_departamento = d.id_departamento
GROUP BY d.id_departamento, d.nombre_departamento
HAVING COUNT(e.id_empleado) >= 5    -- departamentos "significativos"
ORDER BY salario_prom DESC;
```

LIVE CODING

Escribir consultas SQL para obtener la siguiente información:

Tienes una base de datos que registra información sobre empleados y departamentos en una empresa. Tienes dos tablas: "Empleados" y "Departamentos".

1. Obtener los nombres de los empleados y sus salarios que trabajan en el mismo departamento que el empleado con `id_empleado = 3`.
2. Obtener el nombre del departamento y el nombre del empleado con el salario más alto en cada departamento.
3. Obtener el nombre de los departamentos junto con el número de empleados en cada departamento. Ordenar los resultados por la cantidad de empleados en orden descendente.
4. Obtener el nombre del departamento con el mayor promedio de salarios entre los empleados.

Tiempo: 45 minutos

LIVE CODING

Tabla "Empleados":

```
id_empleado (Clave primaria)
nombre
apellido
salario
id_departamento (Clave foránea que se relaciona con id_departamento en la
tabla Departamentos)
```

Tabla "Departamentos":

```
id_departamento (Clave primaria)
nombre_departamento
ubicacion
```

Consultas distintos tipos de JOIN



Consultas con distintos tipos de JOIN

¿Qué son las operaciones JOIN?

Las operaciones JOIN en una consulta SQL son utilizadas para **combinar datos de dos o más tablas** en función de **una relación entre ellas**. Las tablas pueden contener información relacionada, pero dividida en diferentes conjuntos de datos, y las operaciones JOIN permiten unir esos datos para obtener una vista más completa y enriquecida de la información.

Las operaciones JOIN se utilizan principalmente para recuperar información que involucra relaciones entre tablas, como por ejemplo, obtener datos de un cliente junto con sus pedidos o encontrar información sobre estudiantes y los cursos en los que están inscritos.

Consultas con distintos tipos de JOIN

Tipos de operaciones JOIN más utilizados:

INNER JOIN: Combina registros de ambas tablas basados en una condición de coincidencia. Solo los registros que cumplen con la condición se incluyen en el resultado.

LEFT JOIN: Combina todos los registros de la tabla izquierda (primera tabla mencionada) con los registros coincidentes de la tabla derecha (segunda tabla mencionada) según la condición de coincidencia.

RIGHT JOIN : Similar al LEFT JOIN, pero combina todos los registros de la tabla derecha con los registros coincidentes de la tabla izquierda. Si no hay coincidencia, los campos de la tabla izquierda se llenarán con valores NULL.

FULL JOIN: Combina todos los registros de ambas tablas, independientemente de si hay coincidencia o no. Si no hay coincidencia, los campos no coincidentes se llenarán con valores NULL



Consultas con distintos tipos de JOIN

¡Veamos un ejemplo!

```
SELECT Clientes.nombre, Pedidos.numero_pedido  
FROM Clientes  
INNER JOIN Pedidos ON Clientes.id_cliente = Pedidos.id_cliente;
```

En este ejemplo, estamos combinando los datos de las tablas "Clientes" y "Pedidos" utilizando un **INNER JOIN** basado en la coincidencia de la columna "id_cliente". El resultado será una lista de nombres de clientes junto con los números de pedido correspondientes.

Consultas con distintos tipos de JOIN

¡Veamos un ejemplo más complejo!

```
SELECT
    a.nombre, a.apellido,
    COUNT(*) AS libros
FROM Autores a
JOIN Libros l ON l.id_autor = a.id_autor
WHERE l.anio_publicacion >= 2020
GROUP BY a.id_autor, a.nombre, a.apellido
HAVING COUNT(*) >= 3
ORDER BY libros DESC;
```

En este ejemplo, estamos combinando **JOIN** y funciones de agregación como **COUNT(*)** y condiciones de agregación mediante **HAVING**. El resultado será los autores que tengan más de 3 libros publicados posterior a 2020

Consultas complejas

```
SELECT
  e.nombre, e.apellido, d.nombre_departamento, d.ubicacion, e.salario,
  CASE
    WHEN e.salario >= 100000 THEN 'Alto'
    WHEN e.salario >= 70000 THEN 'Medio'
    ELSE 'Bajo'
  END AS rango_salarial
FROM Empleados e
JOIN Departamentos d ON d.id_departamento = e.id_departamento
WHERE
  (
    d.nombre_departamento IN ('Ventas', 'IT')
    AND e.salario > 90000
  )
  OR
  (
    d.ubicacion = 'Chile'
    AND e.salario BETWEEN 60000 AND 80000
  )
ORDER BY d.nombre_departamento, e.salario DESC;
```

En este ejemplo, podemos ver el uso de **CASE**, el cual es utilizado para mostrar valores distintos en base a una o varias condiciones. También podemos ver como el **WHERE** contiene condiciones **AND** y **OR**, para filtrar los resultados

LIVE CODING

Escribir consultas SQL para obtener la siguiente información utilizando JOIN:

Tienes una base de datos que almacena información sobre películas y actores en una plataforma de streaming. Tienes dos tablas: "Películas" y "Actores".

1. Obtener el título de todas las películas junto con el nombre del actor principal en cada una.
2. Obtener el nombre de los actores y el número de películas en las que han trabajado. Ordenar los resultados por el número de películas en orden descendente.
3. Obtener el título de las películas y el nombre del actor principal en películas en las que el actor tenga el mismo género que la película.

Tiempo: 45 minutos



Ejercicio N° 2

Consultas con JOIN

Consultas con JOIN

Contexto: 🙌

Después de dominar las sub-consultas, el siguiente paso es combinar tablas directamente. Practicaremos INNER JOIN y LEFT JOIN para responder preguntas típicas de una biblioteca.

Consigna: 📝

Trabajaremos de nuevo con Libros, Generos, Autores y Editoriales.

1. Catálogo detallado

Muestra titulo, nombre_genero, autor (nombre + apellido) y nombre_editorial de cada libro, uniendo las cuatro tablas con INNER JOIN.

Consultas con JOIN

2. Géneros huérfanos

Lista los géneros que no tienen ningún libro asociado. Usa un LEFT JOIN de Generos contra Libros y filtra donde Libros.id_genero sea NULL.

3. Conteo de libros por autor (incluyendo 0)

Devuelve autor (nombre + apellido) y la cantidad de libros que ha publicado. Deben aparecer también quienes todavía no registran libros (conteo 0). Combina Autores con Libros mediante LEFT JOIN y agrupa.

Tiempo 🕒: 20 minutos

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ Comprender qué son las consultas anidadas
- ✓ Realizar consultas anidadas utilizando SQL
- ✓ Realizar consultas con funciones de agrupación utilizando SQL
- ✓ Realizar consultas con distintos tipos de JOIN



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compártela o tráela al próximo encuentro sincrónico.



#Checkout

¿Qué les pareció la clase de hoy?



< **¡Muchas gracias!** >

